Lambda Calculus
and
Combinatory Algebra

by

Sam L. Speight

Lecture notes and exercises for the

Midlands Graduate School
in the
Foundations of Computing Science
13—17 April 2026
School of Computer Science
University of Nottingham

School of Computer Science
University of Birmingham

Feb–April 2026

Abstract

Lambda calculus and combinatory logic are both theories of *functions*—things that compute or *do*. They are formal systems in which we can program and perform logical reasoning. Superficially, they are a bit different—the main difference being that lambda calculus uses variable binding, whereas combinatory logic does not. Upon further investigation, they are pretty well—though not perfectly—aligned. We will concentrate on the untyped lambda calculus. Although variable binding can be fiddly, lambda calculus has a rather simple and intuitive syntax, which belies its power—both as a model of computation and as an idea. As well as its syntax and connection to combinatory logic, we will study the semantics of the lambda calculus via combinatory algebras.

The idea of combinators came about in the 1920s due to Schönfinkel, who had the intention of reformulating predicate logic without the need for variable binding [Sch24]. Later that decade, Curry rediscovered the idea whilst analyzing substitution. Fun fact: apparently, the idea of Currying also goes back to Schönfinkel; the name stuck despite Curry attributing the idea to Schönfinkel.

Lambda calculus came about later—in the 1930s—thanks to Church [Chu32]. Why 'λ'? In a letter, Church wrote that it was a modification of $\hat{x}$ to $\wedge x$ to λx, where the first of these is used by Russel and Whitehead in *Principia Mathematica* to denote class abstraction. Later in life, he is supposed to have said that λ was chosen essentially at random. For a more thorough history, see [CH09; Sel09]. Lambda calculus (perhaps typed) is the heart of many modern functional programming languages and proof assistants.

Acknowledgements

Three primary sources of inspiration for these lecture notes were Andrew D. Ker's 2009 notes from the *Lambda Calculus and Types* course at Oxford [Ker09], Barendregt's textbook [Bar85] and Selinger's tutorial paper [Sel02].

Gratitude goes to Tom de Jong for the LaTeXtemplate and to Leah Riley for the illustration on the titlepage.

Please contact s.l.speight@bham.ac.uk with any comments or corrections.

Contents

Chapter 1

Introduction

1. My name: Sam Speight. Call me Sam.
2. Lectures: 09:00-09:55 Mon-Fri in A48.
3. Exercise classes: 16:30-17:25 Mon-Thurs in A44.
4. Link to lecture notes: Will be updated throughout the week.
5. Plan for the course.
6. Please do ask questions. Longer discussions can be deferred to exercise classes.

Chapter 2

Syntax of Lambda Calculus

2.1 Terms

Terms of the λ-calculus are certain finite strings of symbols. We have a countably infinite set V of symbols called 'variables'. In addition, we have the symbols '(', ')', '.' and, of course, 'λ' (we could also allow constants, but choose not to).

The set Λ of terms (well-formed strings of symbols) is inductively generated by the following rules. This is to say, Λ is the smallest set of strings of symbols satisfying the following rules.

$$\frac{x \in V}{x \in \Lambda}\ \textsc{var} \qquad \frac{t \in \Lambda \qquad u \in \Lambda}{(tu) \in \Lambda}\ \textsc{app} \qquad \frac{t \in \Lambda \qquad x \in V}{\lambda x.t}\ \textsc{abs}$$

An upshot of this is that we may define functions out of Λ by *recursion* and prove things about all terms terms by *induction*. For both of these, there is one case for each of the above rules.

We adopt the following conventions:

- We drop parentheses with the understanding that application binds tighter than abstraction and application associates to the left. For example, $\lambda x.tuv$ is shorthand for $(\lambda x.((tu)v))$.
- Nested abstractions can be brought under a single λ. For example, $\lambda xy.t$ is shorthand for $\lambda x.\lambda y.t$.

2.2 Variable binding

The variable x in the term $\lambda x.t$ is *bound* by λ. Chances are you've come across bound variables before.

$$\sum_{i=0}^{n} i^2 \qquad \int_0^1 x^2\,\mathrm{d}x \qquad \forall y.P(y)$$

In the summation on the left, the variable i is bound; in the integral in the middle, the variable x is bound; in the logical formula on the right, the variable y is bound.

The *free variables* of a λ-term t are those variables occurring in t that are not bound by a λ. Formally, the set $\mathsf{FV}(t)$ of free variables of a term t is defined by recursion on t.

- $\mathsf{FV}(x) \coloneqq \{x\}$
- $\mathsf{FV}(tu) \coloneqq \mathsf{FV}(t) \cup \mathsf{FV}(u)$
- $\mathsf{FV}(\lambda x.t) \coloneqq \mathsf{FV}(t) - \{x\}$

The final clause dictates that λ binds any occurrences of x in t. A closed term is a term t for which $\mathsf{FV}(t) = \emptyset$. Note that a variable may occur both free and bound in the same term, for example, x in $t \coloneqq x(\lambda x.x)$. $x \in \mathsf{FV}(t)$ because it occurs free in t (as well as occurring bound).

Bound variables are dummy variables in the sense that they can be renamed without changing the content of an expression. For example, the following two summations are the same.

$$\sum_{i=0}^{n} i^2 \qquad \sum_{j=0}^{n} j^2$$

In λ-calculus, we call two terms *α-equivalent* when they differ only in their bound variables. For example, the following two terms are α-equivalent.

$$\lambda x.xz \qquad \lambda y.yz$$

We consider α-equivalent terms to be syntactically identical, which is denoted using $\equiv$. We reserve $=$ for object-level equality, which is weaker.

Moreover, we adopt the 'Barendregt's variable convention':

> In a given mathematical context, we may assume that all bound variables are distinct from each other and from any free variables.

This is possible because terms are finite strings, yet we have a countably infinite supply of variables.

2.3 Substitution

We may substitute a term for a free occurrences of a variable in another term. The result $t[u/x]$ of substituting the term u for free occurrences of x in t is defined recursively as follows.

$$\frac{}{x = x}\ \text{REFL} \qquad \frac{t_1 = t_2}{t_2 = t_1}\ \text{SYM} \qquad \frac{t_1 = t_2 \qquad t_2 = t_3}{t_1 = t_3}\ \text{TRANS}$$

$$\frac{t_1 = t_2 \qquad u_1 = u_2}{t_1 u_1 = t_2 u_2}\ {=}\text{APP} \qquad \frac{t_1 = t_2}{\lambda x.t_1 = \lambda x.t_2}\ \xi$$

$$\frac{}{(\lambda x.t)u = t[u/x]}\ \beta$$

Figure 2.1: The rules for λ-theories

- $x[u/x] :\equiv u$
- $y[u/x] :\equiv y$ (where $y \not\equiv x$)
- $(t_1 t_2)[u/x] :\equiv (t_1[u/x])(t_2[u/x])$
- $(\lambda y.t)[u/x] :\equiv \lambda y.(t[u/x])$

Note that, in the final clause, we do not have to stipulate that $y \not\equiv x$ or $y \notin \mathsf{FV}(u)$ due to Barendregt's variable convention.

Exercise 2.1. Show by induction on s that $t_1[t_2/x][t_3/y] \equiv t_1[t_3/y][t_2[t_3/y]/x]$.

2.4 Lambda theories

Here we introduce equational theories of λ-calculus, or λ-theories. These are particular sets of equations between λ-terms. Such object-level equations are denoted using = (formally, equations are pairs of λ-terms).

Definition 2.2. A λ-theory is a set of equations between λ-terms that is closed under the rules given in Figure 2.1. For a λ-theory T, write $T \vdash t = t'$ when $t = t' \in T$. The unique smallest λ-theory is called $\lambda\beta$.

Technically, each rule in Figure 2.1 specifies a family of pairs, whose first component is a set of equations (the premises), and whose second component is an equation (the conclusion). There may be 'side conditions' on rules (written above the line in parentheses), which cut down the family from what it would otherwise be.

The first row of rules ensure that = is an equivalence relation. The second row of rules says that = respects application and abstraction.

Finally, we have the β-rule. This gives life to λ-calculus. It tells us that abstraction forms functions, application is function application, and the action of a function on its argument is given by substitution. However, there is no formal distinction between functions and data that we feed to functions. For example, we may apply a function to itself! This oddity contributes to the power of the lambda calculus.

Exercise 2.3. Show that $\lambda\beta \vdash \lambda pq.(\lambda xy.y)pq = \lambda ab.a$ by drawing a proof tree.

Closed λ-terms are sometimes called 'combinators'.

Theorem 2.4 (First recursion theorem)**.** *Every λ-term has a fixed point. That is, for every λ-term f there is a λ-term u such that $\lambda\beta \vdash u = fu$.*

Proof. Consider 'Curry's paradoxical combinator': $\mathbf{y} :\equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$. We have $\lambda\beta \vdash \mathbf{y}f = f(\mathbf{y}f)$ (exercise). This means that $\mathbf{y}f$ is a fixed point of f. □

Exercise 2.5. Verify that $\lambda\beta \vdash \mathbf{y}f = f(\mathbf{y}f)$.

We can add rules to those in Figure 2.1 to generate λ-theories. Equations can be considered as rules with no premises. For example, we have the η-equation:

$$\lambda x.tx = t \qquad (x \notin \mathsf{FV}(t)) \tag{η}$$

This makes every term a function. Moreover, we have the following extensionality rule.

$$\frac{t_1 x = t_2 x \qquad (x \notin \mathsf{FV}(t_1) \cup \mathsf{FV}(t_2))}{t_1 = t_2}\ \textsc{ext}$$

This equates two terms that produce the same result when applied to an arbitrary argument.

Exercise 2.6. (i) Show that $\lambda\beta + \eta = \lambda\beta + \{\lambda xy.xy = \lambda x.x\}$.

(ii) Show that $\lambda\beta + \eta = \lambda\beta + \textsc{ext}$.

2.5 List of exercises

1. Exercise 2.1 on nested substitutions
2. Exercise 2.3 on proofs in $\lambda\beta$
3. Exercise 2.5 on fixed points
4. Exercise 2.6 on equivalence of λ-theories

Chapter 3

Reduction

Computation is dynamic; it is a process. In this chapter, we will give dynamics to the lambda calculus by orienting the β-equation. Ultimately, this allows us to deduce consistency of the equational theory $\lambda\beta$.

3.1 Reduction relations

Given a binary relation R on Λ, we define another binary relation $\to_R$ on Λ—the '(one-step) R-reduction' relation—inductively.

$$\frac{(t_1, t_2) \in R}{t_1 \to_R t_2}\ R$$

$$\frac{t_1 \to_R t_2}{t_1 u \to_R t_2 u}\ \textsc{app-left} \qquad \frac{u_1 \to_R u_2}{t u_1 \to_R t u_2}\ \textsc{app-right}$$

$$\frac{t_1 \to_R t_2}{\lambda x.t_1 \to_R \lambda x.t_2}\ \textsc{abs}$$

While $\to_R$ is one step of R reduction, we can also consider many steps of R-reduction and R-equality:

- The reflexive transitive closure of $\to_R$ is denoted by $\twoheadrightarrow_R$. This is obtained by adding a reflexivity rule and a transitivity rule to those above.
- The reflexive symmetric transitive closure of $\to_R$ is denoted $=_R$. This is obtained by adding reflexivity, symmetry and transitivity rules.

3.1.1 Properties of reduction relations

The following notions make sense for any binary relation on a set. However, we are interested in these properties in the context of reduction relations $\to_R$ on Λ. All properties are parametrized by R, though this is sometimes left implicit if R is understood.

- A term $t \in \Lambda$ is in (or is an) *R-normal form* when there is no $t' \in \Lambda$ for which $t \to_R t'$.
- A term $t \in \Lambda$ *has an R-normal form* (or is *weakly R-normalizable*) when there is a normal form $t' \in \Lambda$ such that $t \twoheadrightarrow_R t'$.
- A term $t \in \Lambda$ is *strongly R-normalizable* when there is no infinite sequence $t \to_R t' \to_R t'' \dots$.
- $\to_R$ is *weakly normalizing* when every term has an R-normal form.
- $\to_R$ is *strongly normalizing* when every term is strongly R-normalizable.
- $\to_R$ has the *diamond property* when $r \to_R s_1$ and $r \to_R s_2$ implies $s_1 \to_R t$ and $s_2 \to_R t$ for some t.
- $\to_R$ is *Church-Rosser* when $\twoheadrightarrow_R$ has the diamond property.

Exercise 3.1. Suppose $\to_R$ is Church-Rosser. If $t \in \Lambda$ has an R-normal form, then it is unique.

Proposition 3.2. *If $\to_R$ has the diamond property, then so does $\twoheadrightarrow_R$.*

Proof. By 'diagram chase'. □

Exercise 3.3. Suppose $\to_R$ is Church-Rosser. Show that $t_1 =_R t_2$ if and only if there is some t_3 such that $t_1 \twoheadrightarrow_R t_3$ and $t_2 \twoheadrightarrow_R t_3$. Hint: this is also a diagram chase.

Hence, if two terms are normalizable, then we can check equality by comparing normal forms.

3.2 Beta reduction

The relation we are most interested in is β-reduction, that is:

$$R \coloneqq \{((\lambda x.t)u, t[u/x])\}$$

The following is intuitively clear, but carrying out a formal proof should help solidify understanding.

Exercise 3.4. Show that $=_\beta$ is $\lambda\beta$.

So we can study the equational theory $\lambda\beta$ via the reduction relation $\to_\beta$.

Consider the combinator $\Omega :\equiv (\lambda x.xx)(\lambda x.xx)$. Then there is only one possible β-reduction. That is: $\Omega \to_\beta \Omega$. From this we conclude that the lambda calculus is not even weakly normalizing. However, as we show in the next section, $\to_\beta$ is Church-Rosser. Hence, β-normal forms are unique.

Exercise 3.5. Consider Turing's fixed-point combinator: $\boldsymbol{\theta} :\equiv (\lambda xy.y(xxy))(\lambda xy.y(xxy))$. Show that $f(\boldsymbol{\theta} f) \twoheadrightarrow_\beta \boldsymbol{\theta} f$. Show that not necessarily $f(\mathbf{y}f) \twoheadrightarrow_\beta \mathbf{y}f$.

3.3 Church-Rosser

Theorem 3.6 (Church-Rosser [CR36])**.** *$\to_\beta$ is Church-Rosser.*

The path to proving this result, which we tread in the remainder of this section, is due to Martin-Löf and Tait:

1. Define a notion $\to_{||}$ of 'parallel reduction';
2. Show that $\twoheadrightarrow_\beta$ is the reflexive transitive closure of $\to_{||}$;
3. Show that $\to_{||}$ has the diamond property;
4. Conclude that $\to_\beta$ is Church-Rosser.

The relation $\to_{||}$ of parallel reduction is inductively defined by the following rules.

$$\frac{}{t \to_{||} t}\ \textsc{refl}$$

$$\frac{t_1 \to_{||} t_2 \qquad u_1 \to_{||} u_2}{t_1 u_1 \to_{||} t_2 u_2}\ \textsc{app} \qquad \frac{t_1 \to_{||} t_2}{\lambda x.t_1 \to_{||} \lambda x.t_2}\ \textsc{abs}$$

$$\frac{t_1 \to_{||} t_2 \qquad u_1 \to_{||} u_2}{(\lambda x.t_1)u_1 \to_{||} t_2[u_2/x]}\ ||\beta$$

Exercise 3.7. Show that $\to_\beta \subseteq \to_{||} \subseteq \twoheadrightarrow_\beta$.

Proposition 3.8. *The reflexive transitive closure of $\to_{||}$ is $\twoheadrightarrow_\beta$.*

Proof. From Exercise 3.7 and the fact that taking the reflexive transitive closure of a relation is monotone with respect to $\subseteq$, we have $\twoheadrightarrow_\beta \subseteq \twoheadrightarrow_{||} \subseteq \twoheadrightarrow_\beta$. □

Proposition 3.9. *If $t \to_{||} t'$ and $u \to_{||} u'$, then $t[u/x] \to_{||} t'[u'/x]$.*

Proof. By induction on the derivation of $t \to_{||} t'$.

Case: REFL. As $t' \equiv t$, our goal reduces to showing $t[u/x] \to_{||} t[u'/x]$. We do a further induction on the structure of t.

Subcase: VAR. We have two cases: $t \equiv x$ and $t \equiv y$ for some variable $y \not\equiv x$. The first case amounts to showing $u \to_{||} u'$, which holds by assumption. The second case amounts to showing $y \to_{||} y$, which holds by REFL.

Subcase: $t = t_1 t_2$. This amounts to showing $t_1[u/x]t_2[u/x] \to_{||} t_1[u'/x]t_2[u'/x]$, which holds by the induction hypothesis.

Subcase: $t \equiv \lambda y.s$. Amounts to showing $\lambda y.(s[u/x]) \to_{||} \lambda y.(s[u'/x])$, which holds by induction hypothesis.

Case: APP. Exercise.

Case: ABS. Exercise.

Case: $||\beta$. We have to show that $((\lambda y.t_1)u_1)[u/x] \to_{||} (t_2[u_2/y])[u'/x]$, assuming $t_1 \to_{||} t_2$ and $u_1 \to_{||} u_2$. The induction hypothesis is $t_1[u/x] \to_{||} t_2[u'/x]$ and $u_1[u/x] \to_{||} u_2[u'/x]$. We calculate:

$$\begin{aligned}
((\lambda y.t_1)u_1)[u/x] &\equiv ((\lambda y.t_1)[u/x])(u_1[u/x]) \\
&\to_{||} t_2[u'/x][u_2[u'/x]/y] && \text{(by IH and } ||\beta) \\
&\to_{||} (t_2[u_2/y])[u'/x] && \text{(by Exercise 2.1)}
\end{aligned}$$

□

Exercise 3.10. Fill in the APP and ABS cases from the proof of Proposition 3.9.

Lemma 3.11. *$\to_{||}$ has the diamond property.*

Proof. We show that $r \to_R s_1$ and $r \to_R s_2$ implies $s_1 \to_R t$ and $s_2 \to_R t$ for some t by induction on the derivation of $r \to_R s_1$. We just give two cases.

Case: REFL. As $s_1 \equiv r$, take $t \equiv s_2$.

Case: APP. The final step in the derivation of $r \to_R s_1$ can have one of two forms:

$$\frac{p \to_{||} p_1 \qquad q \to_{||} q_1}{pq \to_{||} p_1 q_1}\ \text{APP} \qquad\qquad \frac{\lambda x.p \to_{||} \lambda x.p_1 \qquad q \to_{||} q_1}{(\lambda x.p)q \to_{||} (\lambda x.p_1)q_1}\ \text{APP}$$

If we are in the left case, then the last step in the derivation of $r \to_{||} s_2$ has the form:

$$\frac{p \to_{||} p_2 \qquad q \to_{||} q_2}{pq \to_{||} p_2 q_2}\ \text{APP}$$

By the induction hypothesis, we get t_1, t_2 such that $p_1 \to_{||} t_1$, $p_2 \to_{||} t_1$, $q_1 \to_{||} t_2$ and $q_2 \to_{||} t_2$. Take $t \equiv t_1 t_2$. Then $p_1 q_1 \to_{||} t$ and $p_2 q_2 \to_{||} t$ by the APP rule, as required.

If we are in the right case, then the last step in the derivation of $r \to_{||} s_2$ has the form:

$$\frac{p \to_{||} p_2 \qquad q \to_{||} q_2}{(\lambda x.p)q \to_{||} x[q_2/p_2]}\ \text{APP}$$

By the induction hypothesis, we get t_1, t_2 such that $p_1 \to_{||} t_1$, $p_2 \to_{||} t_1$, $q_1 \to_{||} t_2$ and $q_2 \to_{||} t_2$. Take $t \equiv x[t_2/t_1]$. Then $s_1 \equiv (\lambda x.p)q \to_{||} t$ by the $||\beta$ rule, and $s_2 \equiv p_2[q_2/x] \to_{||} t$ by Proposition 3.9. □

Exercise 3.12. Complete the remaining cases in the proof of Lemma 3.11.

This completes the proof of Theorem 3.6, that $\to_\beta$ is Church-Rosser.

3.4 Consistency

We turn our focus back to λ-theories. What does it mean for a λ-theory to be consistent or inconsistent? We don't have any (native) connectives like $\neg$ (negation) and $\wedge$ (and). Well, we should be able to deduce interesting consequences from a consistent theory. That is, consistent theories should be informative. Turning this around, an inconcsistent theory should be completely uninformative. This idea is formalized in the following definition.

Definition 3.13. A λ-theory T is inconsistent when for all $t, t' \in \Lambda$, $T \vdash t = t'$. A λ-theory T is consistent when it is not inconsistent, ie. there exist terms t, t' such that $T \nvdash t = t'$.

In the next chapter, we will encode Booleans (true and false) as λ-terms. Another reasonable definition of inconsistency is that a theory equates true and false. We will see that this is equivalent to the above definition.

Theorem 3.14. *$\lambda\beta$ is consistent.*

Proof. Take any two distinct normal forms s, s'. Then $\lambda\beta \vdash s = s'$ iff $s =_\beta s'$ iff there is some t such that $s \twoheadrightarrow_\beta$ and $s' \twoheadrightarrow_\beta t$. But there is no such t, as s, s' are in normal form. □

Terms without a normal form are computations that never terminate. Can we lump all such terms together (in an 'undefined' category)?

Theorem 3.15. *$T_{\mathsf{NF}} \coloneqq \lambda\beta + \{t = t' \mid t, t' \text{ closed and not weakly normalizable}\}$ is inconsistent.*

Proof. If r is a closed term, then the following are closed terms.

$$\lambda xy.xr \qquad \lambda xy.yr$$

are in normal form. Then for arbitrary s, s':

$$\begin{aligned} T_{\mathsf{NF}} \vdash s &= (\lambda xy.x)sr \\ &= (\lambda xy.xr)((\lambda xy.x)s)((\lambda xy.x)s') \\ &= (\lambda xy.yr)((\lambda xy.x)s)((\lambda xy.x)s') \\ &= (\lambda xy.x)s'r \\ &= s' \end{aligned}$$

□

3.5 List of exercises

1. Exercise 3.1 on Church-Rosser and unique normal forms
2. Exercise 3.3 on equality and reduction

3. Exercise 3.4 on the relationship between β-reduction and the equational theory $\lambda\beta$
4. Exercise 3.5 on Turing's fixed-point combinator
5. Exercise 3.7 on inclusion of reduction relations
6. Exercise 3.10 on parallel reduction and substitution
7. Exercise 3.12 on the diamond property for $\rightarrow_{||}$

Bibliography

[Bar85] Hendrik Pieter Barendregt. *The lambda calculus - its syntax and semantics.* Vol. 103. Studies in logic and the foundations of mathematics. North-Holland, 1985. ISBN: 978-0-444-86748-3 (cit. on p. ii).

[CH09] Felice Cardone and J. Roger Hindley. "Lambda-Calculus and Combinators in the 20th Century". In: *Logic from Russell to Church.* Ed. by Dov M. Gabbay and John Woods. Vol. 5. Handbook of the History of Logic. North-Holland, 2009, pp. 723–817. DOI: https://doi.org/10.1016/S1874-5857(09)70018-4. URL: https://www.sciencedirect.com/science/article/pii/S1874585709700184 (cit. on p. i).

[Chu32] Alonzo Church. "A Set of Postulates for the Foundation of Logic". In: *Annals of Mathematics* 33.2 (1932), pp. 346–366. ISSN: 0003486X, 19398980. URL: http://www.jstor.org/stable/1968337 (visited on 04/13/2026) (cit. on p. i).

[CR36] Alonzo Church and J. B. Rosser. "Some properties of conversion". In: *Transactions of the American Mathematical Society* 39.3 (1936), pp. 472–482. DOI: 10.1090/S0002-9947-1936-1501858-0 (cit. on p. 8).

[Ker09] Andrew D. Ker. *Lambda Calculus and Types.* Lecture notes, University of Oxford. 2009. URL: https://www.cs.ox.ac.uk/andrew.ker/docs/lambdacalculus-lecture-notes-ht2009.pdf (cit. on p. ii).

[Sch24] M. Schönfinkel. "Über die Bausteine der mathematischen Logik". In: *Mathematische Annalen* 92 (1924), pp. 305–316. URL: http://eudml.org/doc/159074 (cit. on p. i).

[Sel02] Peter Selinger. "The lambda calculus is algebraic". In: *Journal of Functional Programming* 12.6 (2002), pp. 549–566. DOI: 10.1017/S0956796801004294 (cit. on p. ii).

[Sel09] Jonathan P. Seldin. "The Logic of Church and Curry". In: *Logic from Russell to Church.* Ed. by Dov M. Gabbay and John Woods. Vol. 5. Handbook of the History of Logic. North-Holland, 2009, pp. 819–873. DOI: https://doi.org/10.1016/S1874-5857(09)70019-6. URL: https://www.sciencedirect.com/science/article/pii/S1874585709700196 (cit. on p. i).